

Multi-Agent AI Framework for Task-Aware Model Routing and Execution

DeepStackers Team Members

Oruç Çakır

Computer Engineering

Mehmet Gür

Artificial Intelligence Engineering

Serin Evcim

Artificial Intelligence Engineering

Ömer İlbilgi

Computer Engineering

Ulaş Kaya

Computer Engineering



OUTLINE



PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

Why Current AI Workflows Break Down



Heterogeneous User Tasks

Modern users do not need only chat. They need code understanding, web research, document work, data analysis, and visual interpretation within a single workflow.



Fragmented Tools and Context Loss

These tasks are usually split across separate tools and assistants, forcing users to switch context, repeat intent, and lose workflow continuity.



No Controlled Task Specialization

Single-agent systems process diverse requests through one general reasoning path, without a reliable mechanism to match each task with the right expertise.

PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

Main Agent Orchestration & Framework Goals



Core Mission

Build a production-ready orchestration framework where a central **MainAgent** routes user requests to specialized AI agents through a single conversational interface.



System Capabilities

- 9 specialized agents
- 12 LLM provider support
- Real-time streaming
- RAG-based code search
- Full auth and persistence

PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

Main Agent Orchestrating 9 Specialized Agents



GitHub Agent



Codebase Agent



Google Workspace Agent



Search Agent



Vision Agent



Data Analyst Agent



Coding Agent



Frontend Agent



HuggingFace Agent

PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

Existing Solutions in Literature



Multi-Agent Orchestration

CAMEL, ChatDev, AutoGen: Focus on inter-agent collaboration, role assignment, and chat-based task management.



Tool Use & Reasoning

ReAct, ToolLLM, HuggingGPT: Enabling agents to invoke external APIs through a continuous "thought + action" loop.



Optimization & RAG

- llama.cpp, FlexGen: Local/edge device efficiency.
- Vector Databases: RAG & HNSW-based semantic search.

Gaps in the Literature



Vendor Lock-in

Many existing solutions are strictly tied to a single API ecosystem (e.g., exclusively OpenAI), limiting flexibility.



The "Black Box" Problem

Internal decision-making and task delegation processes are hidden from the user, usually only accessible via post-execution logs.



Static Architecture

Adding a new agent typically requires significant, hard-coded modifications to the core orchestration engine.

Our Solution & Key Differentiators

1. Hybrid Model Orchestration

Our custom LLM Factory architecture allows seamless, task-based coordination between Cloud APIs (Gemini, GPT-4) and Local Inference Engines (Ollama, LM Studio) simultaneously.

2. Real-Time Execution Transparency

Proprietary NDJSON Streaming Protocol streams LangGraph decision moments to UI. Users watch the agent's "Thinking Process" and execution trace live.

3. "Self-Registration" Agent Architecture

Plug-and-play module loading system. New specialized agents dynamically register into the orchestrator (MainAgent) at runtime without altering core routing logic.

4. Code-Specific Autonomous RAG

AST-aware (Function/Class-based) chunking for source code. Combined with pgvector and HNSW indexing for highly accurate architectural context retrieval.

OUTLINE



PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

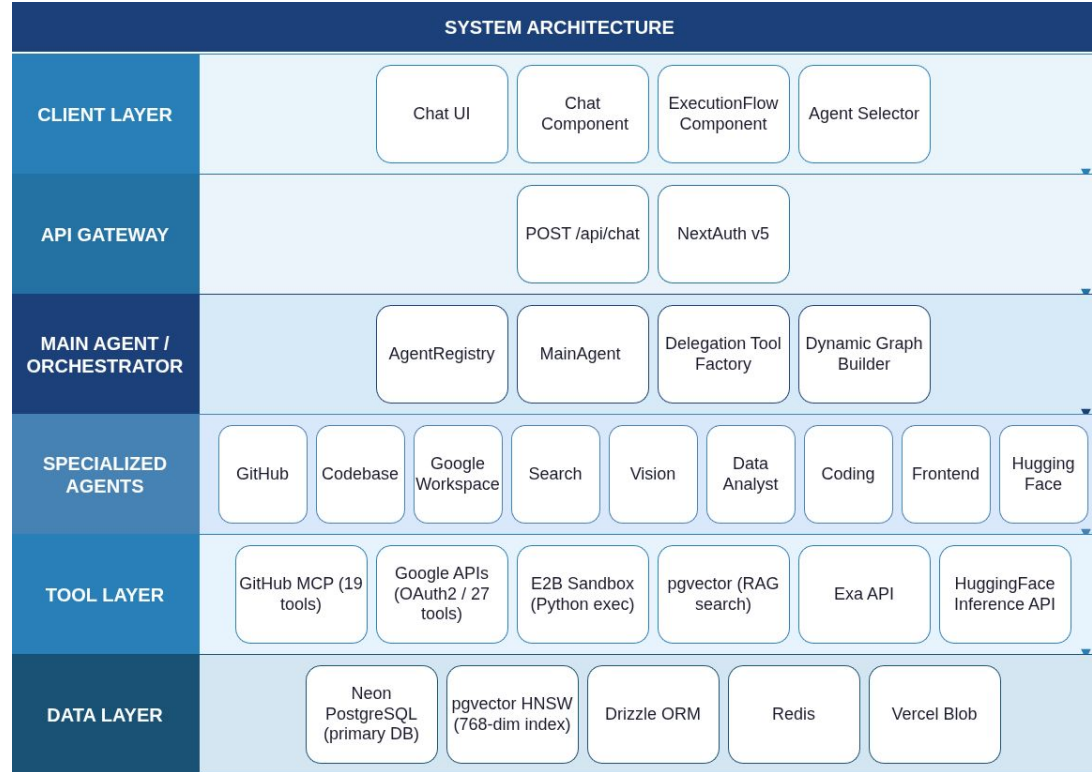
RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

METHOD & DESIGN



Methodological Framework Components

1. Orchestration

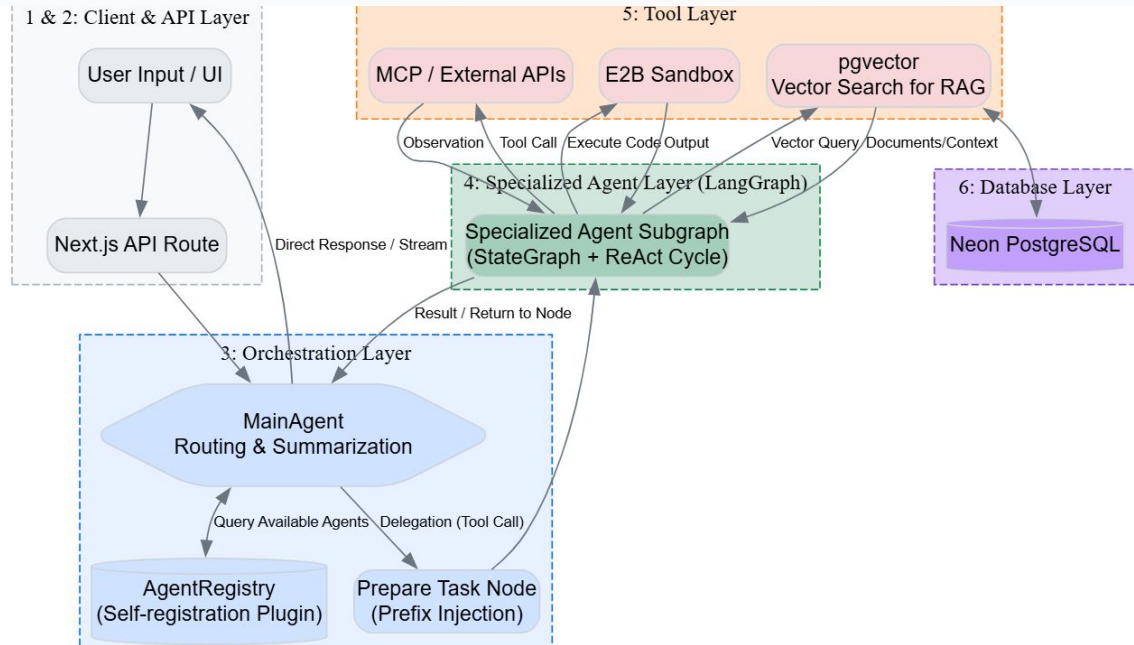
Delegation-as-a-Tool Orchestration for seamless agent task management.

2. Architecture

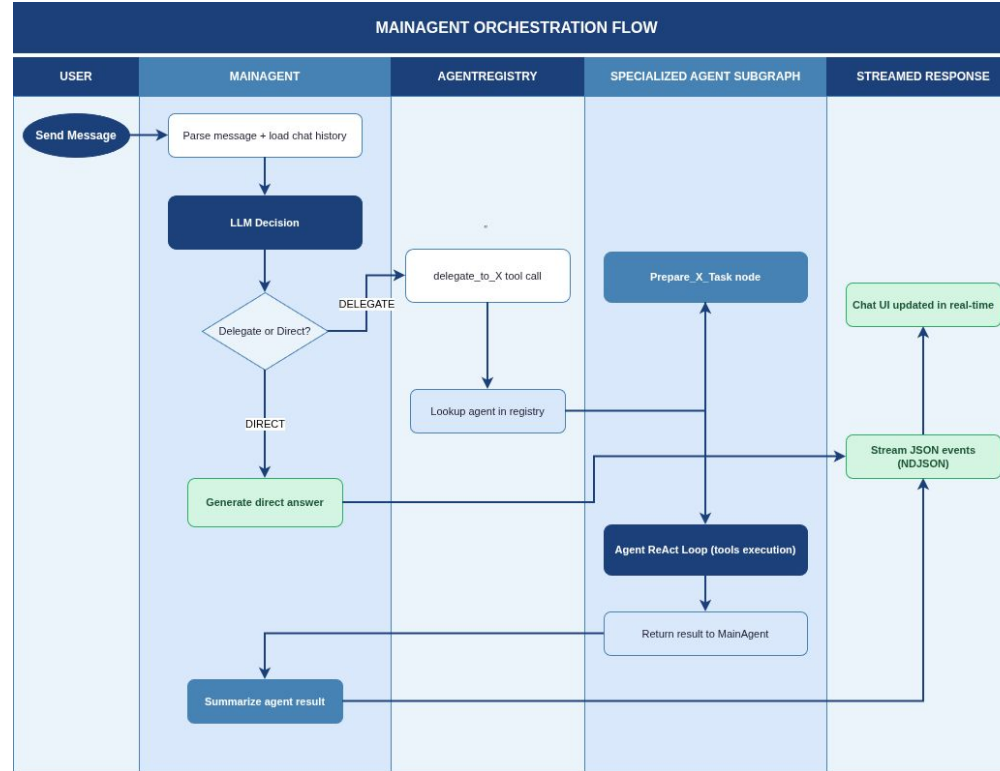
Self-Registration Plugin Architecture for modular scalability.

3. Cycles & Data

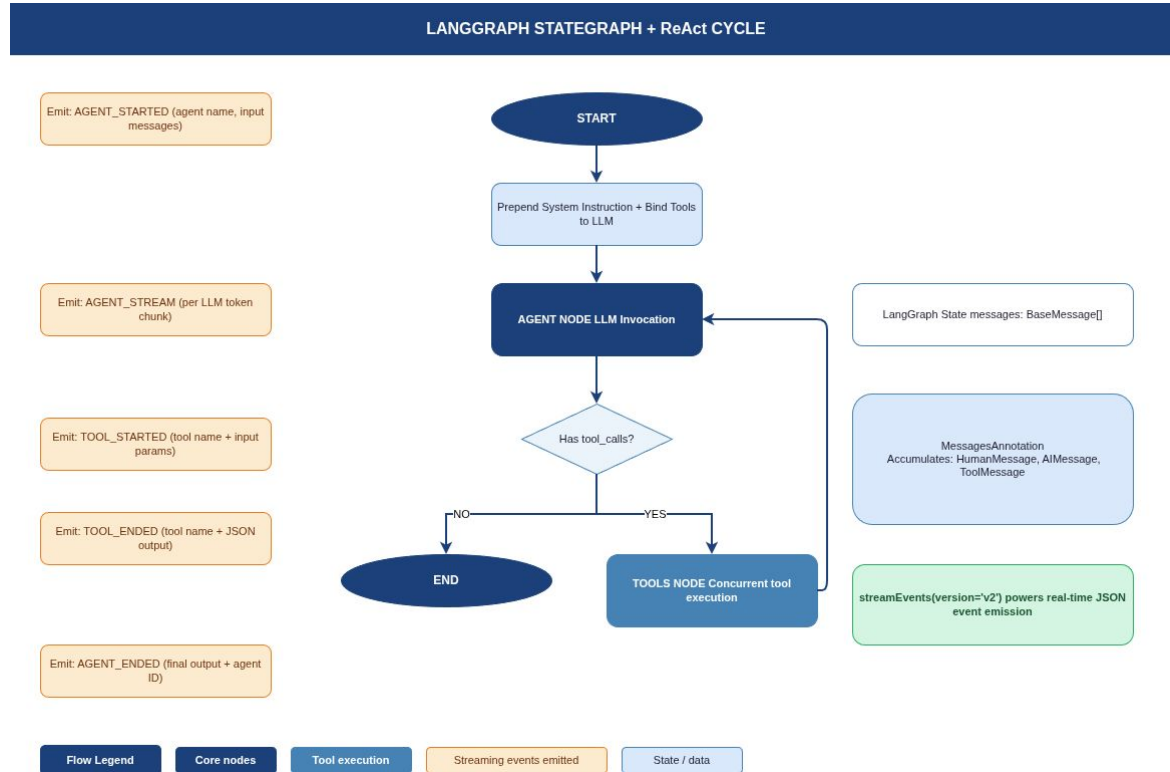
Stateful ReAct Cycles combined with efficient RAG Implementation.



METHOD & DESIGN



METHOD & DESIGN



PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

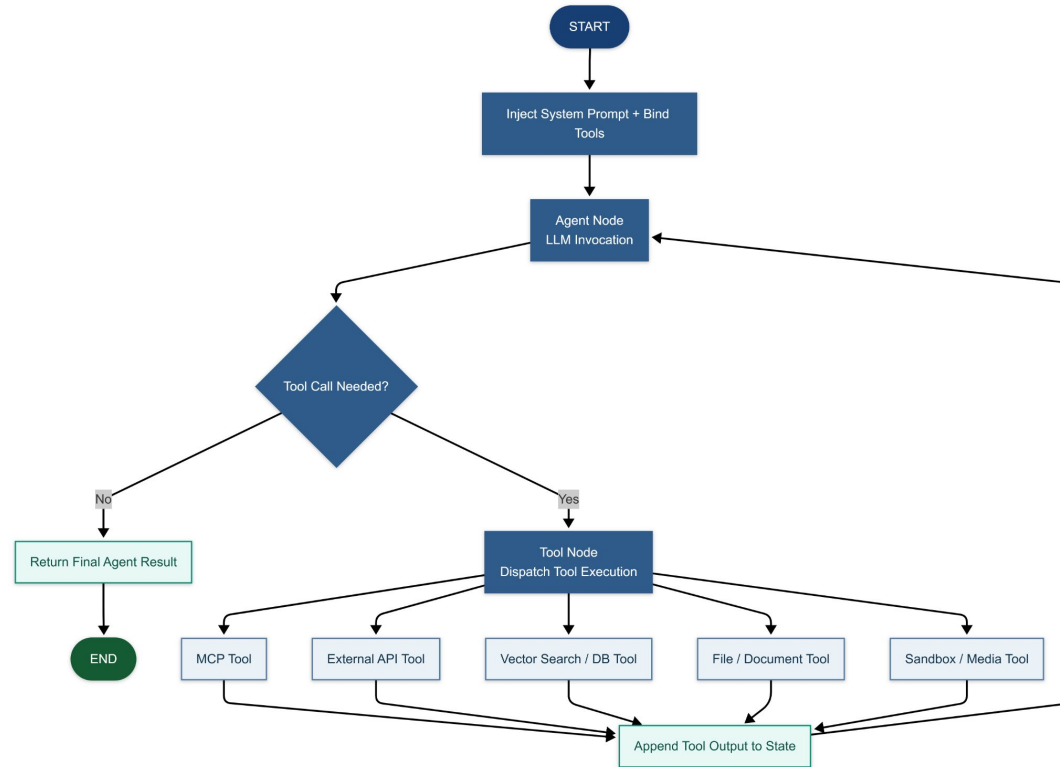
RESULTS

Key findings and task-routing efficiency.

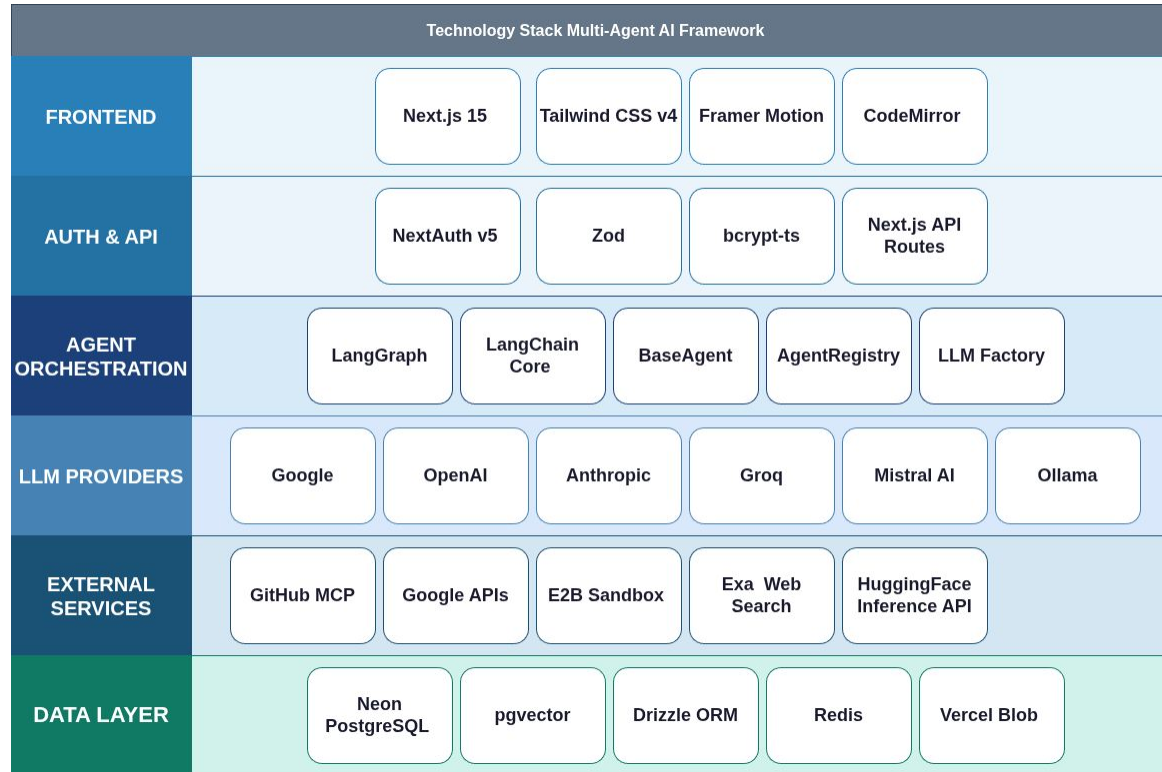
FUTURE WORK & CONCL.

Next steps and final wrap-up.

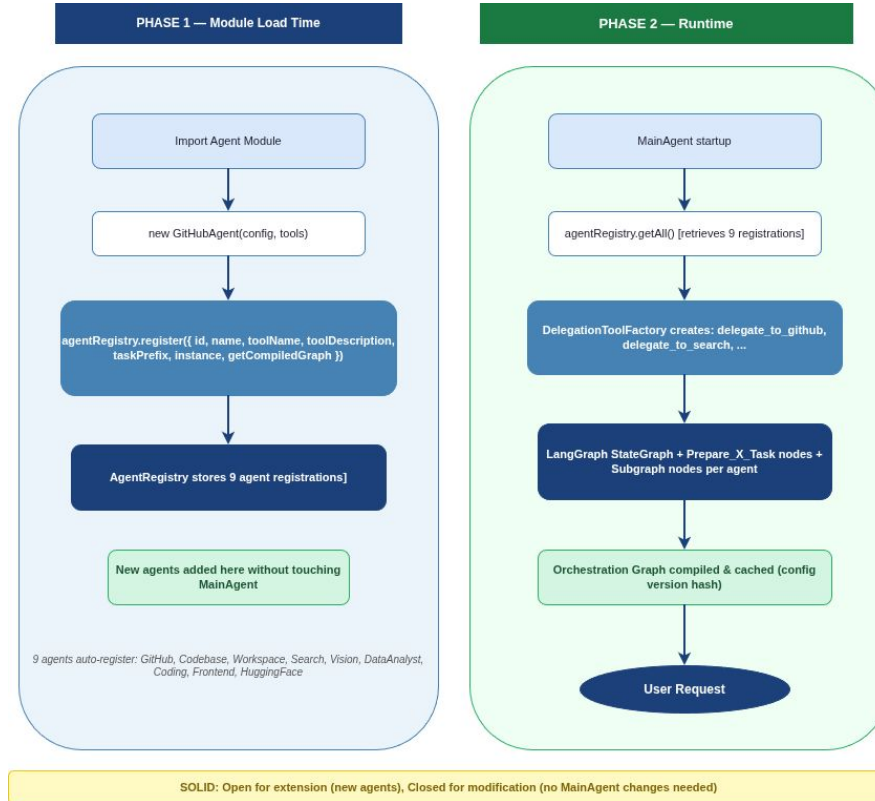
IMPLEMENTATION



IMPLEMENTATION



IMPLEMENTATION





PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.



491

automated tests

61

test files

Unit

251

Logic, agents, tools, utilities, encryption

Integration

46

API routes, authorization, validation

System

43

Complete user journeys (E2E)

User

43

Session, auth, guest flows, entitlements

Performance

29

Latency, throughput, scalability

TESTING & EVALUATION



Unit Tests

Vitest

complete isolation with mocked external dependencies

- Configuration resolution (cloud vs self-hosted)
- Agent registry and tool management
- Encryption and security utilities
- Core utilities and helper functions

Integration

Playwright

proper communication and interactions between modules

- API route validation and error handling
- Authentication and authorization checks
- User ownership and data isolation
- Document and artifact CRUD operations

Performance

Playwright

measure speed, responsiveness, and scalability

- Streaming latency (TTFB / TTLC)
- API route response times
- History query scalability
- Encryption throughput under load

System

Playwright

end-to-end application flow from browser to backend, simulates real user interactions

- Full chat message and streaming flow
- Artifact creation and interaction
- Message editing and reasoning display
- Multi-user session and navigation

User

Playwright

user-focused testing primarily through E2E scenarios

- Guest and registered session flows
- Login, registration and logout
- User quota and access restrictions
- Overall UI and chat experience



PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

Deliverables

- Completed a full-scale, production-grade Multi-Agent AI Framework for intelligent task orchestration.
- Integrated 9 specialized agents and 88+ specialized tools.
- Incorporated 12 different LLM providers, spanning cloud and self-hosted environments.
- Developed real-time NDJSON streaming and Execution Flow tree interface.

Success Status

- Established a 5-layer validation architecture: Unit, Integration, E2E, Route, and Performance.
- Executed 486 unique automated tests across 54 validated test suites.
- Achieved a 100% overall pass rate in the final verification cycle.

Key Strengths

- **Provider Agnosticism:** Eliminated vendor lock-in via a custom LLM Factory for cloud/local models.
- **Extensibility:** "Self-Registration" architecture for adding agents at module load time.
- **Structural RAG Pipeline:** AST-aware chunking with pgvector and HNSW indexing.

OUTLINE



PROBLEM DEFINITION

Identifying core challenges in current AI systems.

OBJECTIVE

Defining the primary goals of the framework.

PROJECT SCOPE

Boundary and deliverables of the project.

LITERATURE REVIEW

Current state of Multi-Agent AI research.

METHOD & DESIGN

Technical architecture and routing logic.

IMPLEMENTATION

Real-world execution and setup details.

TESTING & EVALUATION

Performance metrics and benchmarking.

RESULTS

Key findings and task-routing efficiency.

FUTURE WORK & CONCL.

Next steps and final wrap-up.

Future Enhancements & Roadmap

Broader Orchestration

Extending integration validation to specialized agents, ensuring all delegation paths are tested.

Cross-Browser & Mobile

Expanding UI validation beyond desktop to include Firefox, WebKit, and mobile environments.

Performance Tracking

Storing time-series benchmarks to track TTFC, total latency, and config costs over time.

Resilience & Fault-Injection

Simulating provider failures, timeouts, and degraded third-party dependencies.

Advanced Security

Testing token-expiry, malformed sessions, and adversarial interaction combinations.

Conversation Durability

Long-duration soak testing to ensure stability across resumed or edited conversations.

Conclusion

Production-Grade Architecture

The Multi-Agent AI Framework represents a fully realized, enterprise-grade system designed for intelligent and complex task orchestration.

Modern Engineering Excellence

The system's architecture strongly exemplifies modern software-engineering practices, including abstraction, composition, extensibility, and strict type safety.

Decoupled and Flexible

The framework successfully decouples individual agent implementation from core orchestration concerns, enabling runtime flexibility through intelligent configuration cascading.

Seamless Extensibility

It provides a robust foundation for integrating new specialized agents without requiring any system-wide modifications to the core engine.

Deployment-Ready Scalability

The system is fully ready for deployment, capable of handling multi-disciplinary tasks through intelligent agent routing and state management.

REFERENCES



- [1] G. Li et al., "CAMEL: Communicative Agents for 'Mind' Exploration of Large Language Model Society," arXiv preprint arXiv:2303.17760, 2023.
- [2] C. Qian et al., "ChatDev: Communicative Agents for Software Development," arXiv preprint arXiv:2307.07924, 2023.
- [3] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint arXiv:2308.08155, 2023.
- [4] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv preprint arXiv:2210.03629, 2022.
- [5] Y. Qin et al., "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs," arXiv preprint arXiv:2307.16789, 2023.
- [6] Y. Shen et al., "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face," arXiv preprint arXiv:2303.17580, 2023.
- [7] G. Gerganov, "llama.cpp," GitHub Repository, 2023.
- [8] Y. Sheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," arXiv preprint arXiv:2303.06865, 2023.
- [9] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv preprint arXiv:2005.11401, 2020.
- [10] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, 2018.